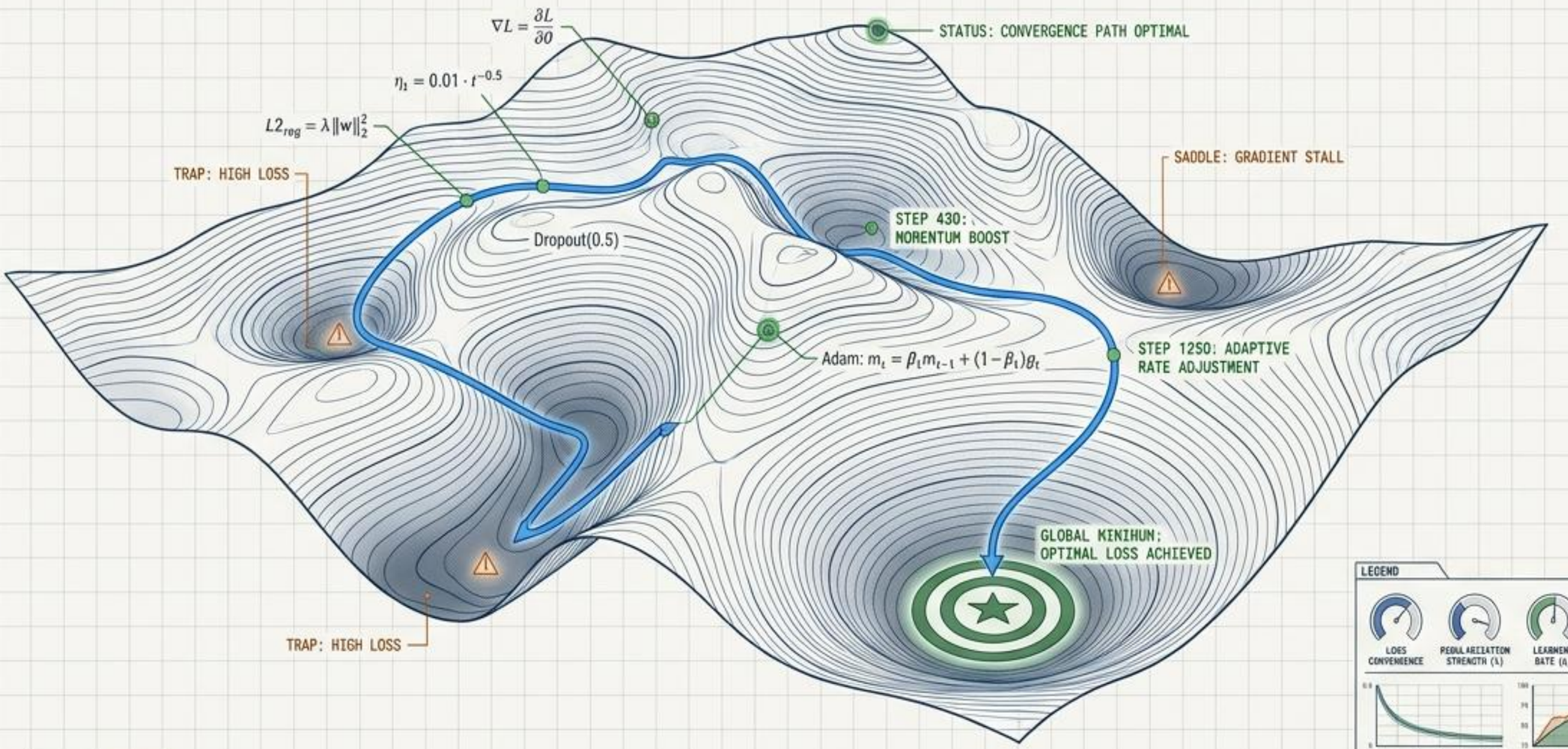
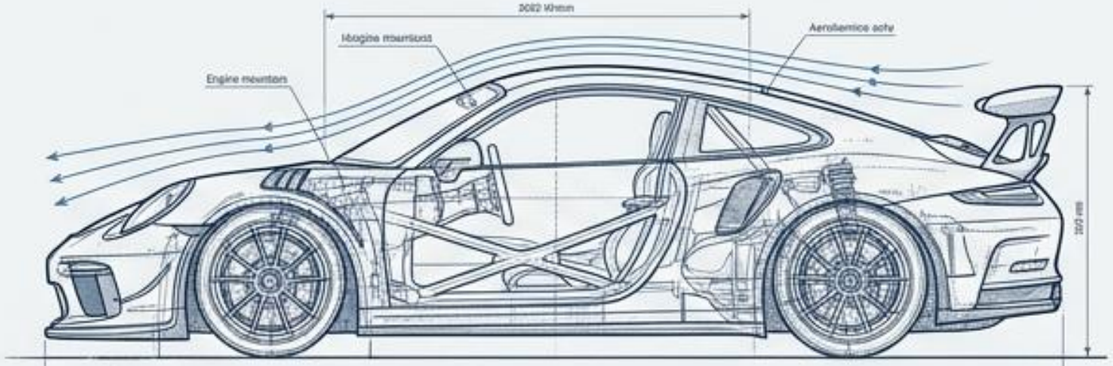
OPTIMIZACIÓN Y REGULARIZACIÓN AVANZADA

Estrategias de Entrenamiento para Redes Neuronales Profundas con TensorFlow/Keras

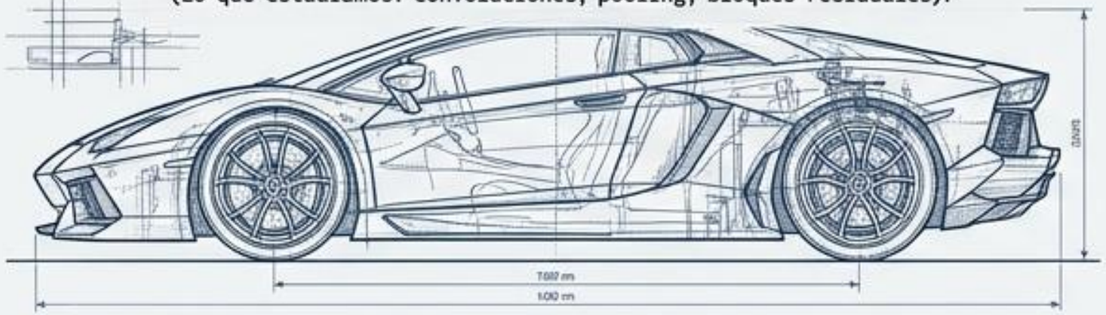


La Arquitectura vs. El Entrenamiento



Diseñar la red es construir el auto.

(Lo que estudiamos: convoluciones, pooling, bloques residuales).



El Problema Central



Optimizar es saber conducirlo.

(Cómo alcanzar el máximo rendimiento sin estrellarse).

Los 5 Pilares

1. Optimizador



(El volante)

2. Tasa de Aprendizaje



(Acelerador)

3. Regularización



(Los frenos)

4. Inicialización



(Posición de salida)

5. Estrategia Keras



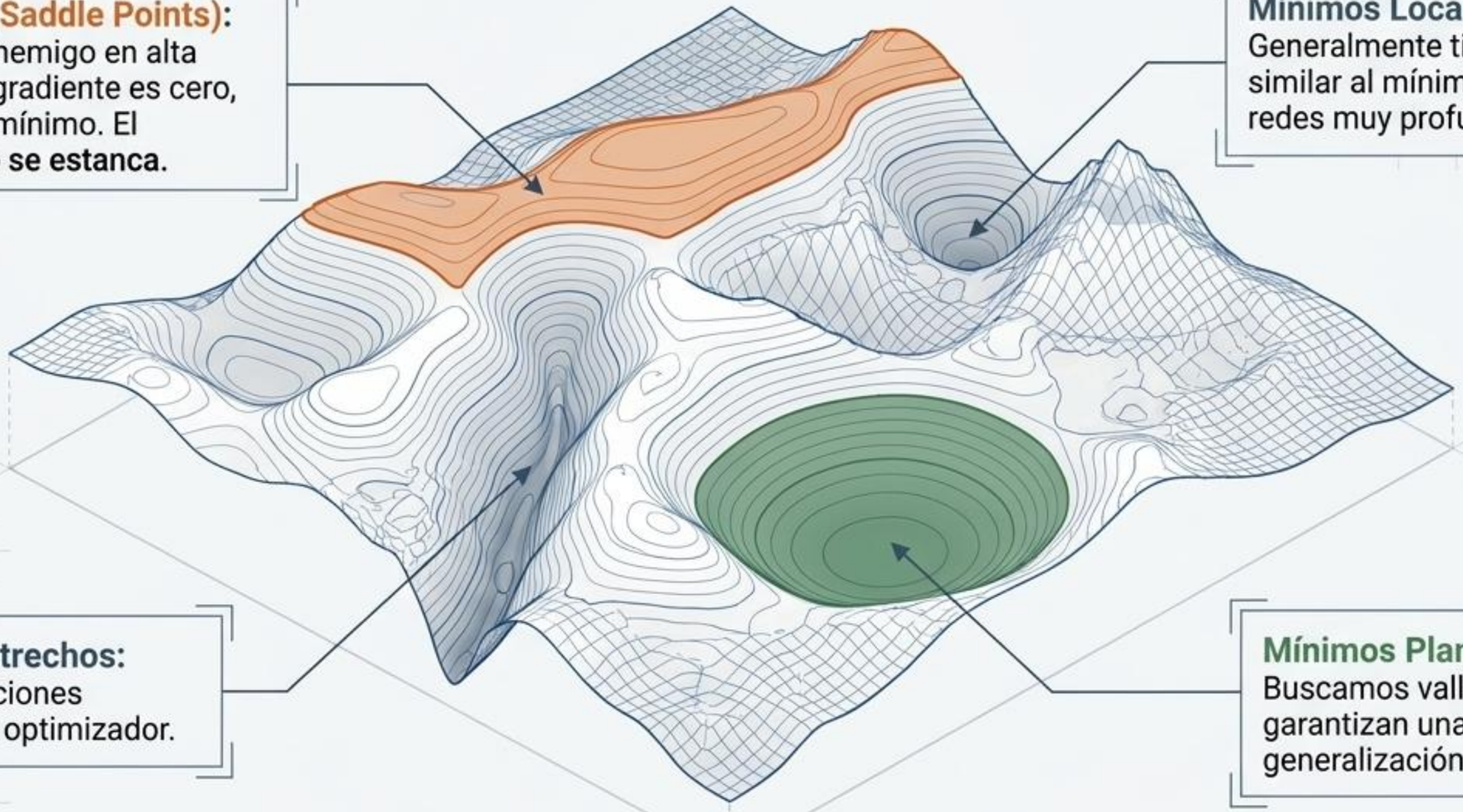
(Telemetría)

Cartografía de la Función de Pérdida

Navegando el espacio de millones de dimensiones

Puntos Silla (Saddle Points):
El verdadero enemigo en alta dimensión. El gradiente es cero, pero no es un mínimo. El **entrenamiento se estanca.**

Mínimos Locales:
Generalmente tienen un loss similar al mínimo global en redes muy profundas.

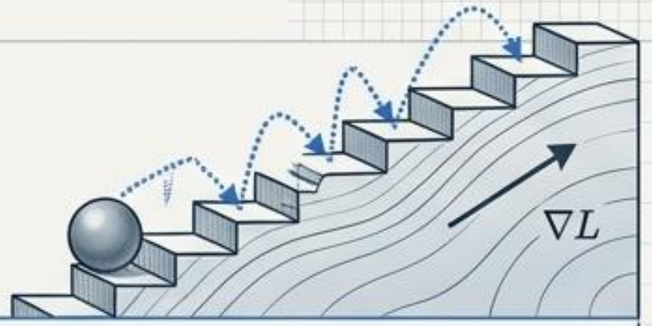


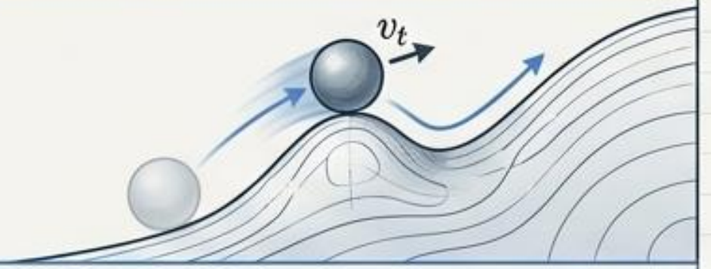
Barrancos Estrechos:
Causan oscilaciones violentas en el optimizador.

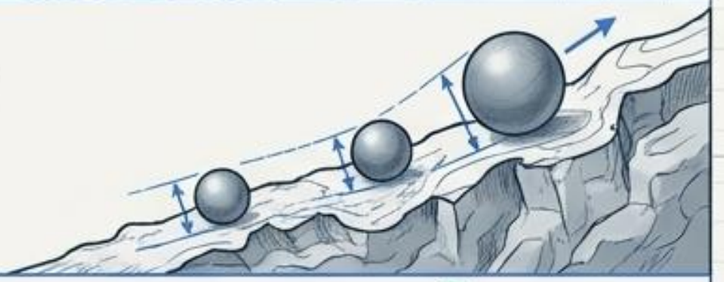
Mínimos Planos:
Buscamos valles anchos: garantizan una mejor generalización.

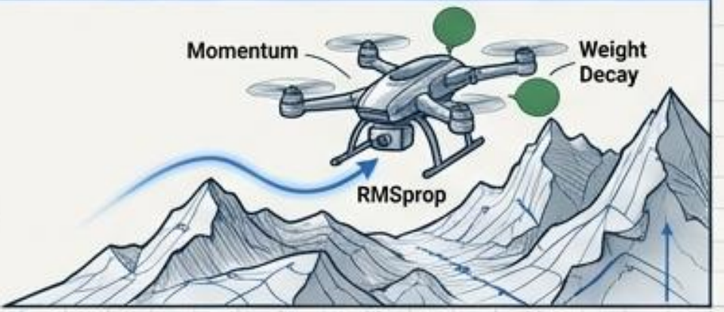
Evolución de los Optimizadores

SGD	$\theta_{t+1} = \theta_t - \eta \cdot \nabla L$	Avanza según el gradiente actual. Ruidoso y propenso a estancarse.
SGD + Momentum	$v_t = \beta \cdot v_{t-1} + \eta \cdot \nabla L$	**Acumula inercia** para escapar de mínimos locales y puntos silla.
RMSprop	División por $\sqrt{E[g^2]_t}$	**Adapta la tasa de aprendizaje** por cada parámetro usando gradientes recientes.
Adam & AdamW	`Momentum` + `RMSprop` + Desacople L2	**AdamW:** El estándar actual. Desacopla el Weight Decay para una mejor generalización.









Tasa de Aprendizaje: El 'Punto Dulce'

Muy Bajo: Estancamiento | Adecuado | Muy Alto: Divergencia



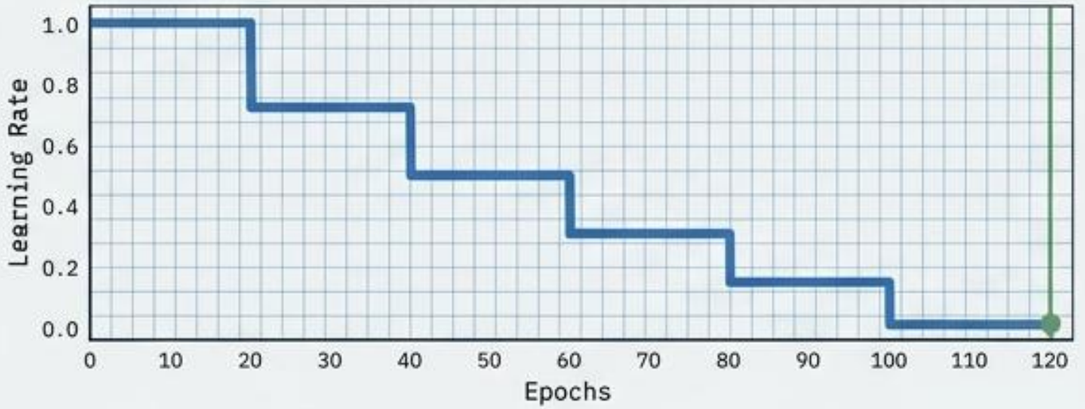
El Método LR Finder (Leslie Smith)

Incrementar el LR exponencialmente durante 1-2 épocas. Elegir el punto donde el loss desciende más rápido (antes de la explosión).

```
lr = min_lr * (max_lr/min_lr)**(1/steps)
if loss > 4 * initial_loss:
    stop_training = True
```

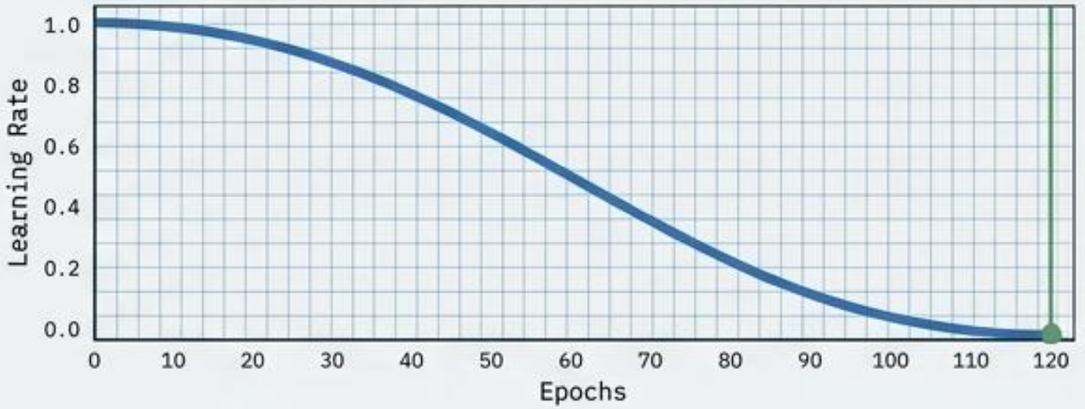
LR Scheduling: Programación Dinámica

Step Decay



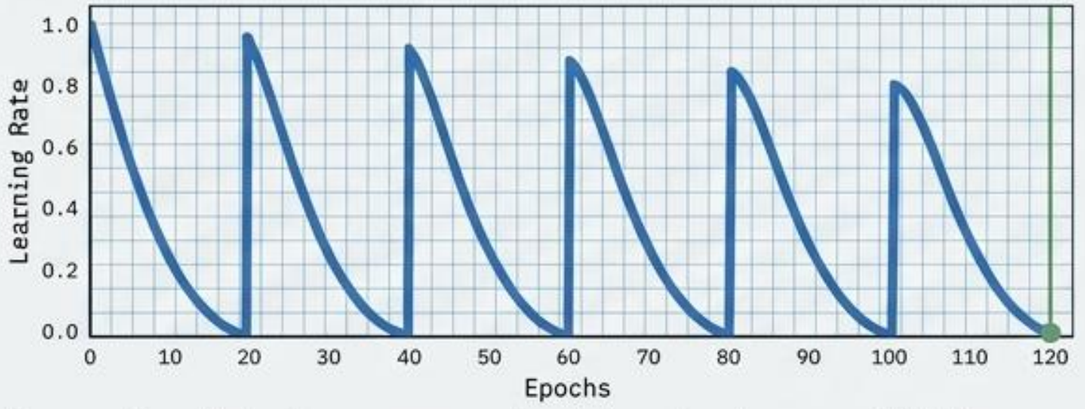
Líneas base y reproducibilidad. Caídas bruscas en el loss.

Cosine Annealing



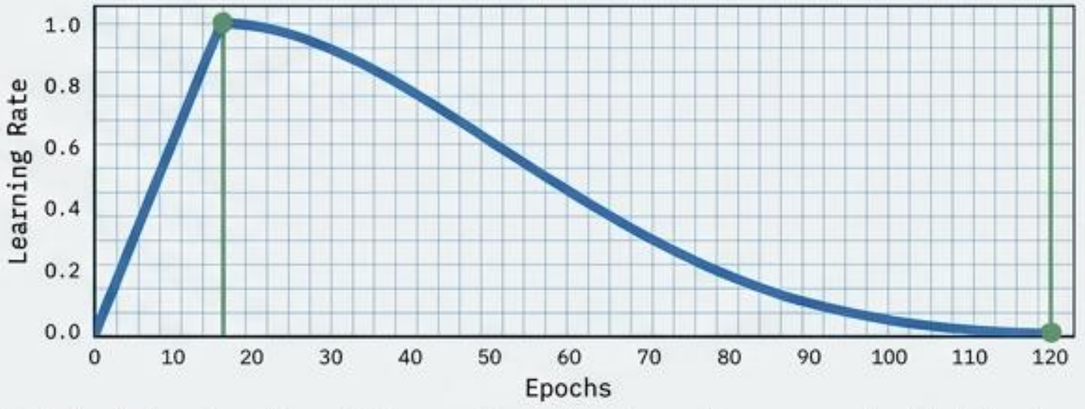
Transición suave de exploración a refinamiento fino.

Warm Restarts (SGDR)



Fuerza al optimizador a escapar de mínimos locales superficiales.

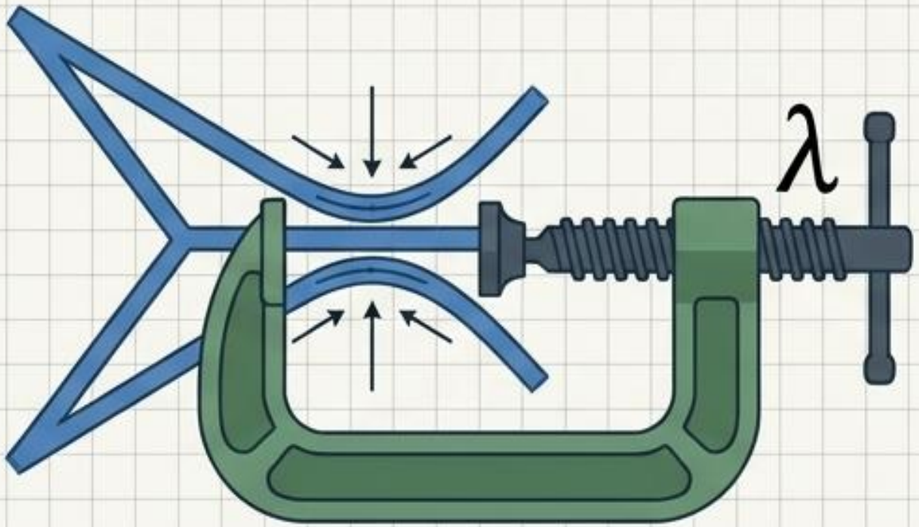
Warmup + Cosine Decay



Estado del arte. Esencial para Transfer Learning y grandes batch sizes.

Frenos y Guardarraíles I: Evitando la Memorización

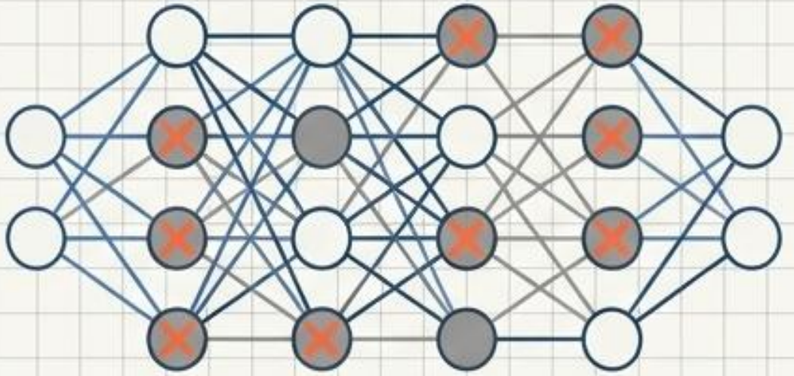
Weight Decay (L2)

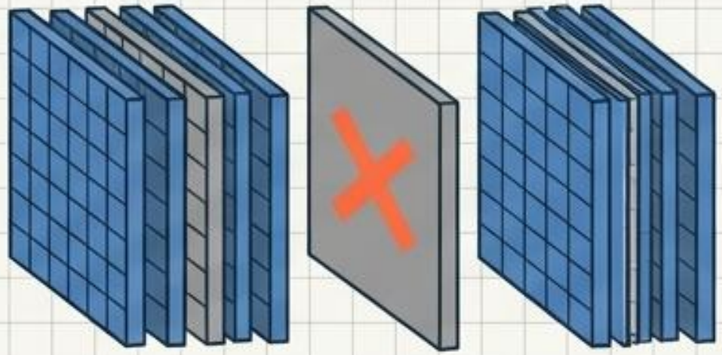


$$L_{total} = L_{orig} + \lambda \sum w^2$$

Penaliza pesos grandes. Fuerza a la red a usar múltiples características en lugar de depender excesivamente de una sola.

Dropout & Spatial Dropout





Dropout previene la co-adaptación. Spatial Dropout2D desactiva canales completos, vital para CNNs donde píxeles vecinos están muy correlacionados.

Frenos y Guardarraíles II: Data Augmentation Avanzada

Imagen Base



Cutout



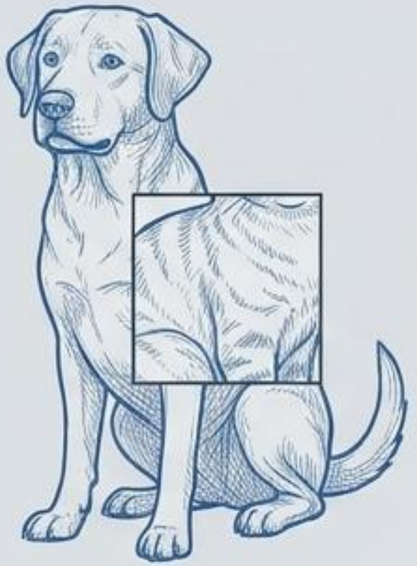
Fuerza a la red a no depender de características obvias y buscar contexto.

Mixup



Combina linealmente imágenes y etiquetas. Crea transiciones suaves.

CutMix

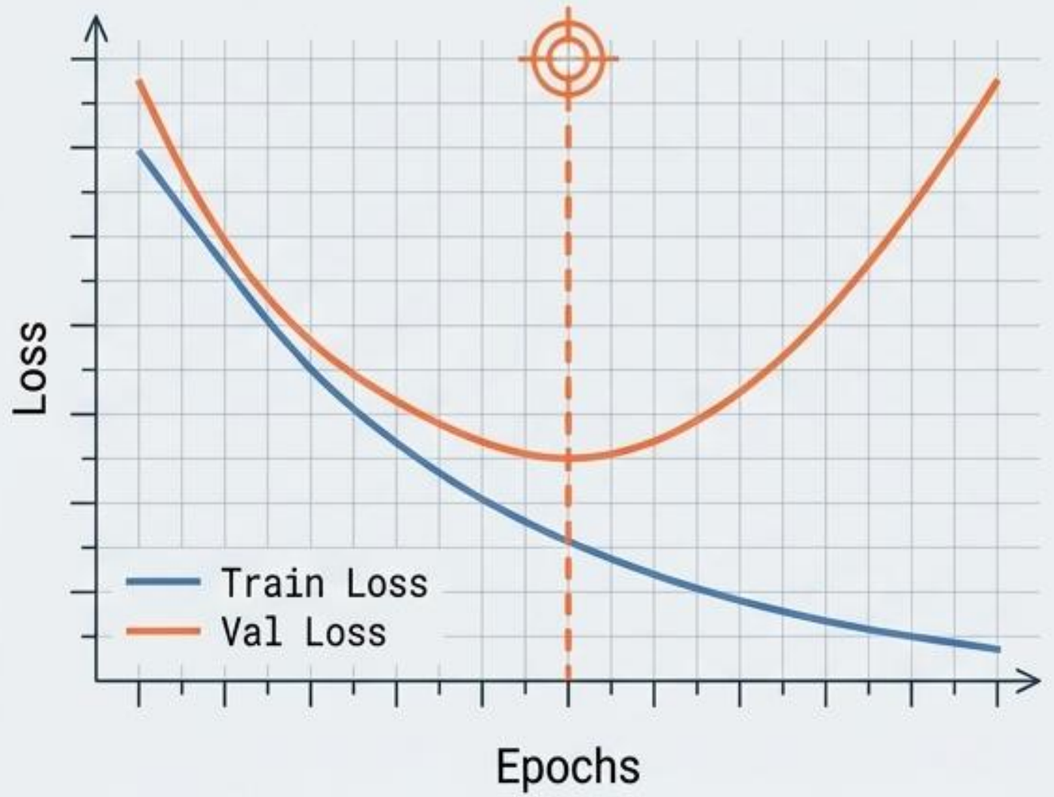


Ajusta las etiquetas proporcionalmente al área del parche insertado.

Nota: Batch Normalization también actúa como regularizador implícito inyectando ruido estadístico por mini-batch.

El Guardián del Tiempo y la Certeza

Early Stopping



El momento óptimo de extracción. Monitorear val_loss con **patience=10**. Detener antes de memorizar ruido.

Label Smoothing



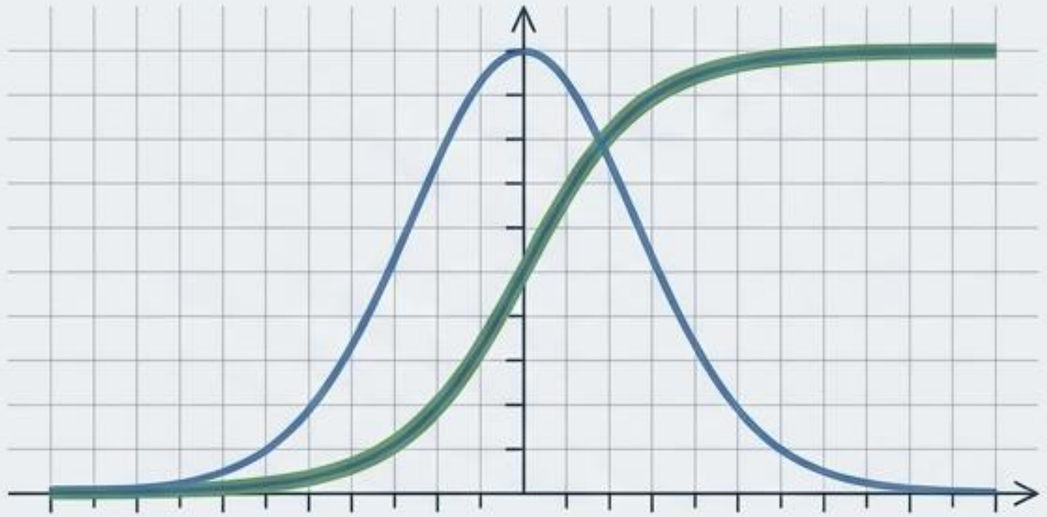
Evita logits extremos. Reduce el exceso de confianza de la red, mejorando la calibración y reduciendo el overfitting.

La Línea de Salida: Inicialización de Pesos

¿Por qué **no inicializar en ceros**? El problema de **simetría**: todas las neuronas calcularían lo mismo y la red perdería expresividad.

Inicialización Xavier / Glorot

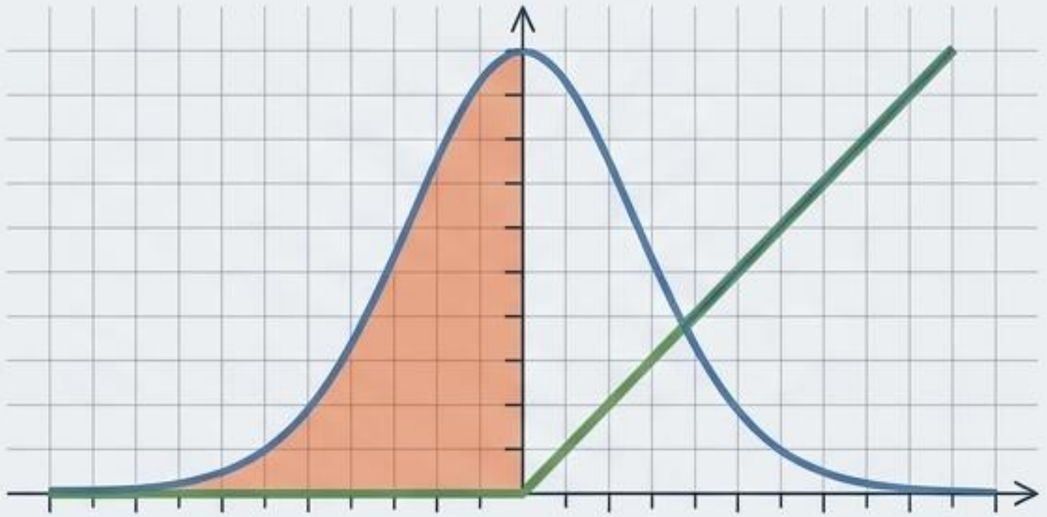
$$N(0,\ 2\ /\ (n_in + n_out))$$



Diseñada para activaciones simétricas. Mantiene la varianza constante a través de las capas.

Inicialización He / Kaiming

$$N(0,\ 2\ /\ n_in)$$



Diseñada para ReLU. Compensa el descarte del 50% de activaciones duplicando la varianza inicial.

Estrategias de Alto Rendimiento (El Motor Keras)

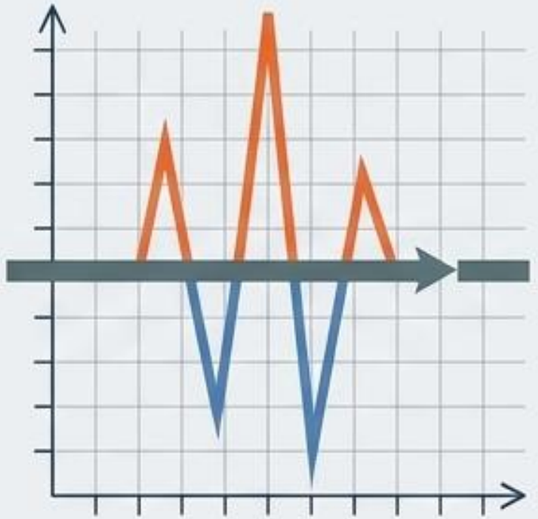
Batch Size & LR Scaling



Regla lineal: Si aumentas el batch size por un factor K, el Learning Rate debe aumentar por K. Requiere Warmup.

```
batch=256 -> lr=0.08
```

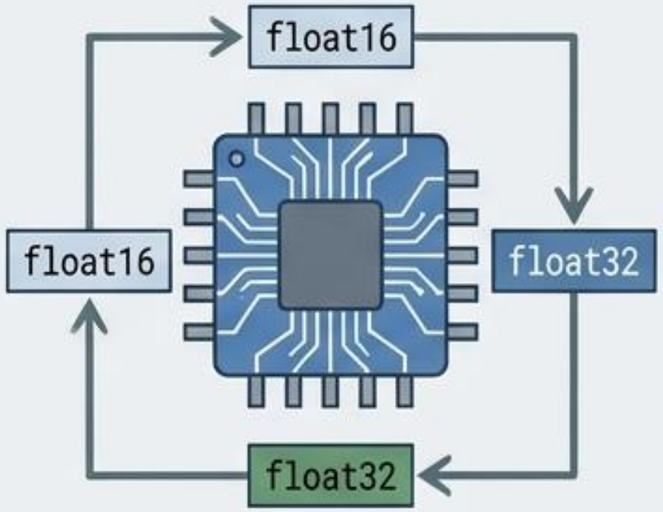
Gradient Clipping



Previene la explosión de gradientes limitando su magnitud máxima. Vital en redes profundas.

```
optimizer=Adam(..., clipnorm=1.0)
```

Mixed Precision

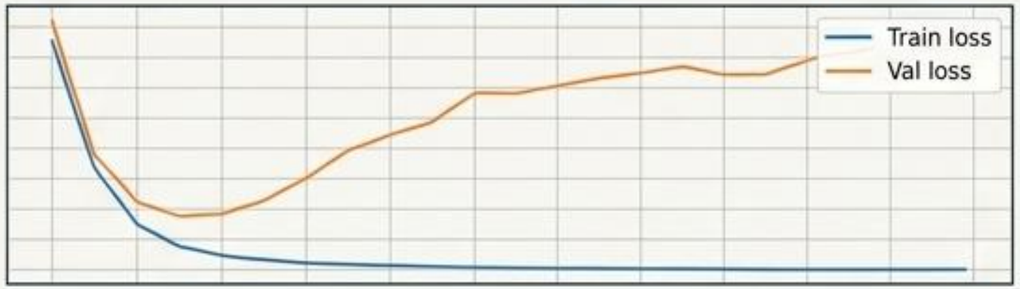


Computación en float16, actualización en float32. ~2x más rápido en GPUs modernas, mitad de memoria VRAM.

```
set_global_policy('mixed_float16')
```

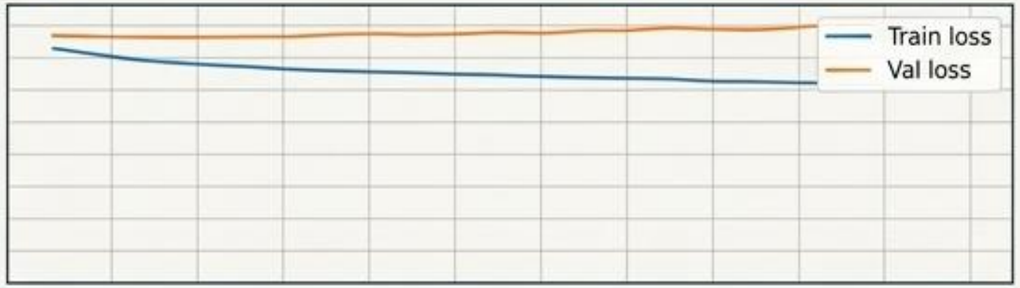
Panel de Diagnóstico: Interpretación de Telemetría

Diagnóstico: Overfitting (Sobreajuste)



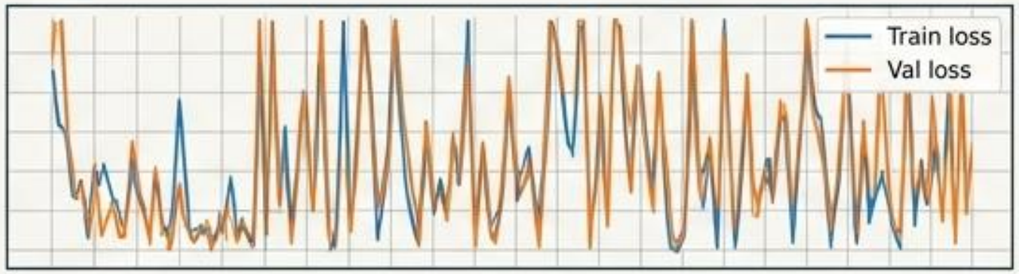
Acción: Aumentar L2/Dropout, aplicar Data Augmentation, usar Early Stopping.

Diagnóstico: Underfitting (Subajuste)



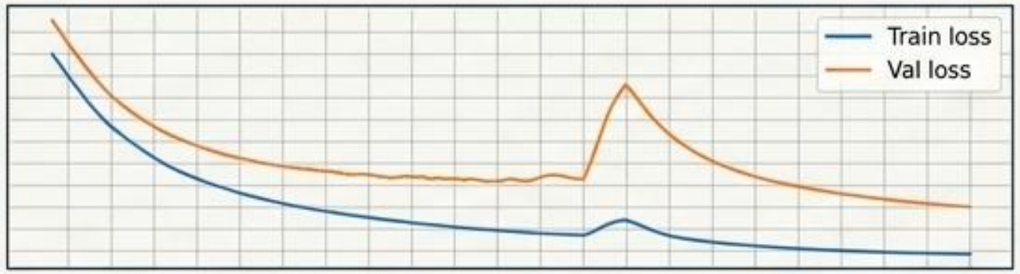
Acción: Aumentar complejidad del modelo, reducir regularización, subir LR.

Diagnóstico: Inestabilidad (LR muy alto)



Acción: Reducir Learning Rate drásticamente, activar Gradient Clipping.

Diagnóstico: Normal (Scheduling activo)



Acción: Comportamiento esperado con Warm Restarts. Continuar entrenamiento.

Receta 1: Entrenando una CNN desde Cero

1. Datos



Aplicar RandomFlip, Rotation, Zoom.

2. Arquitectura Base



Conv2D (He Init) -> BatchNorm -> ReLU -> SpatialDropout

3. Optimizador & Scheduling



AdamW (weight_decay=1e-4) + Cosine Decay + Warmup. Label Smoothing ε=0.1

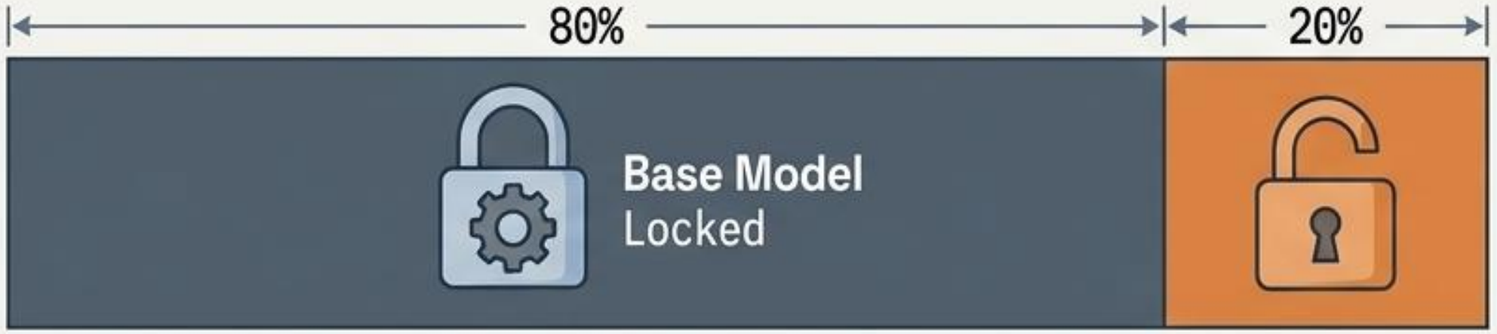
4. Callbacks



EarlyStopping (patience=15) + ModelCheckpoint (guardar max val_acc)

Receta 2: Estrategia de Transfer Learning

Fase 1: “Feature Extraction (Extracción)”





LR = 1e-3

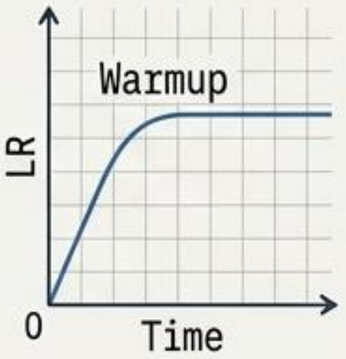
Congelar modelo base (trainable = False). Entrenar solo la nueva cabeza de clasificación.

Fase 2: “Fine-Tuning (Ajuste Fino)”



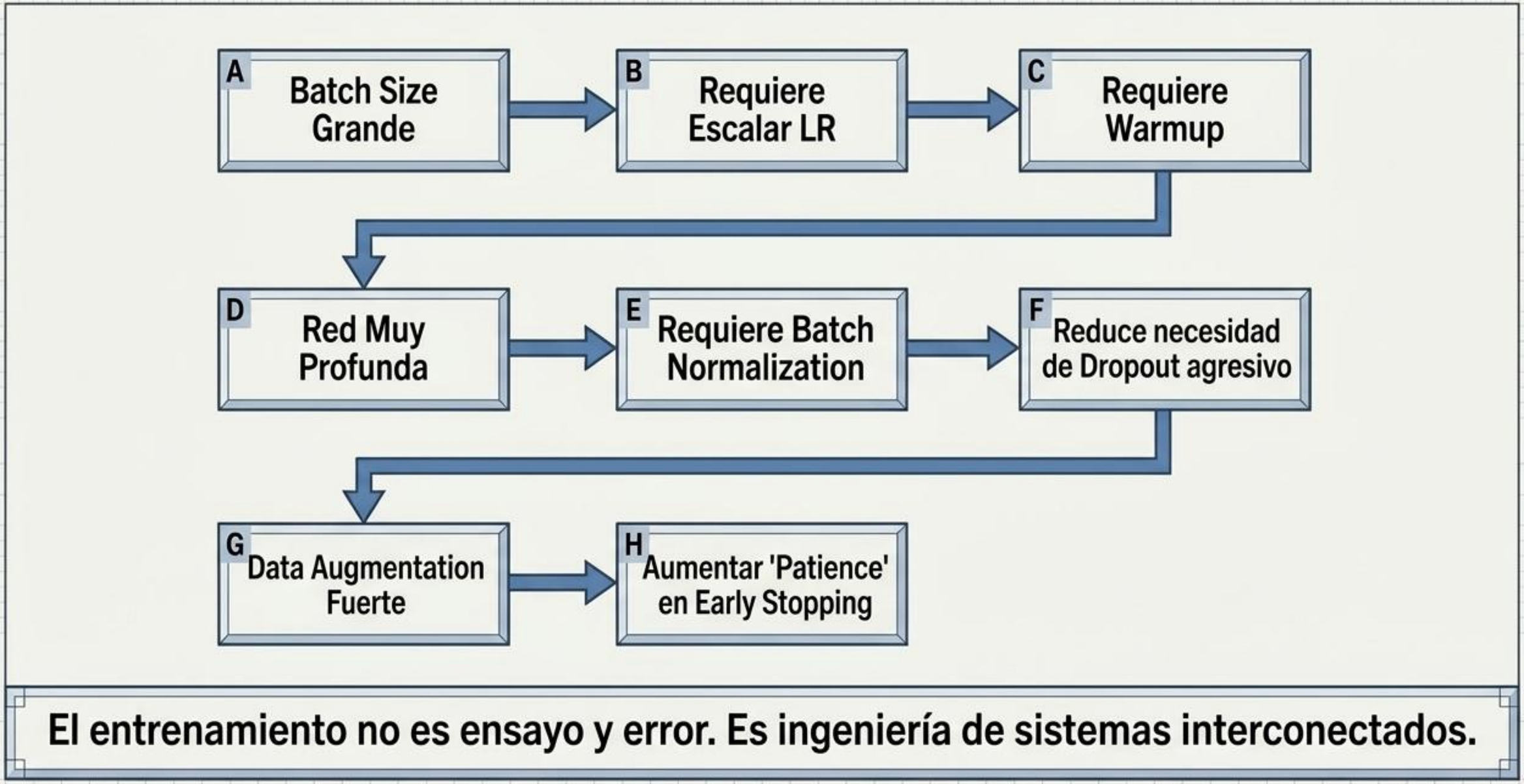


LR = 1e-5



Descongelar capas superiores del modelo base. Usar Learning Rate mínimo + Warmup estricto para no destruir los pesos pre-entrenados.

El Mapa de Decisiones: Ecosistema de Entrenamiento



El entrenamiento no es ensayo y error. Es ingeniería de sistemas interconectados.